Lab 8 · Captures des tests

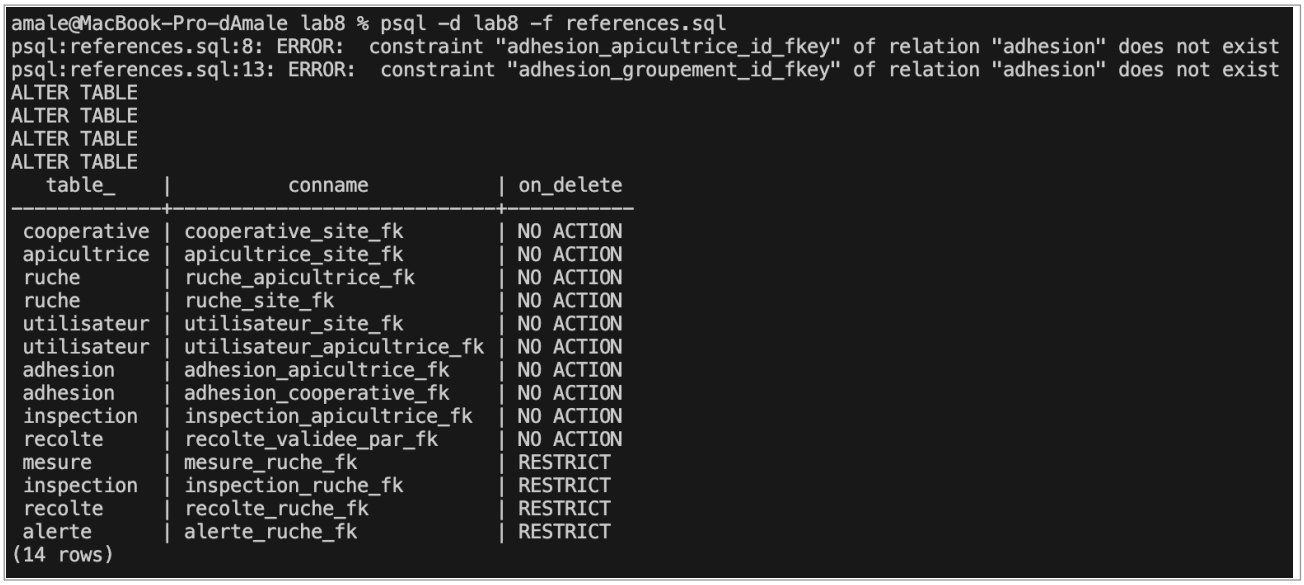
Contraintes, trigger d'audit et rôles · base lab8 (PostgreSQL 17)

Ce document regroupe les captures d'écran du Lab 8, rangées par étape : clés étrangères (8.1), contraintes CHECK (8.2), trigger d'audit (8.3), rôles (8.4) et tests de contournement (8.6).

Étape 8.1 · Que se passe-t-il quand on supprime ?

Figure 1 · references.sql et liste des clés étrangères

```
psql -d lab8 -f references.sql
```



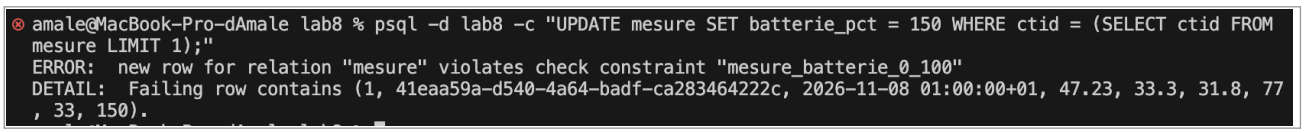
```
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -f references.sql
psql:references.sql:8: ERROR:  constraint "adhesion_apicultrice_id_fkey" of relation "adhesion" does not exist
psql:references.sql:13: ERROR:  constraint "adhesion_groupement_id_fkey" of relation "adhesion" does not exist
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
   table_    |          conname           | on_delete
-------------+----------------------------+-----------
 cooperative | cooperative_site_fk        | NO ACTION
 apicultrice | apicultrice_site_fk        | NO ACTION
 ruche       | ruche_apicultrice_fk       | NO ACTION
 ruche       | ruche_site_fk              | NO ACTION
 utilisateur | utilisateur_site_fk        | NO ACTION
 utilisateur | utilisateur_apicultrice_fk | NO ACTION
 adhesion    | adhesion_apicultrice_fk    | NO ACTION
 adhesion    | adhesion_cooperative_fk    | NO ACTION
 inspection  | inspection_apicultrice_fk  | NO ACTION
 recolte     | recolte_validee_par_fk     | NO ACTION
 mesure      | mesure_ruche_fk            | RESTRICT
 inspection  | inspection_ruche_fk        | RESTRICT
 recolte     | recolte_ruche_fk           | RESTRICT
 alerte      | alerte_ruche_fk            | RESTRICT
(14 rows)
```

Les ALTER TABLE sur mesure, inspection, recolte et alerte passent (RESTRICT). Les deux contraintes de adhesion n'ont pas été trouvées sous les noms utilisés (la table contient adhesion_apicultrice_fk et adhesion_cooperative_fk), elles restent donc en NO ACTION.

Étape 8.2 · Refuser les valeurs impossibles

Figure 2 · Test d'une batterie à 150 %

```
UPDATE mesure SET batterie_pct = 150 WHERE ctid = (SELECT ctid FROM mesure LIMIT
1);
```



```
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -c "UPDATE mesure SET batterie_pct = 150 WHERE ctid = (SELECT ctid FROM
mesure LIMIT 1);"
ERROR:  new row for relation "mesure" violates check constraint "mesure_batterie_0_100"
DETAIL:  Failing row contains (1, 41eaa59a-d540-4a64-badf-ca283464222c, 2026-11-08 01:00:00+01, 47.23, 33.3, 31.8, 77
, 33, 150).
```

Refusé par la contrainte mesure_batterie_0_100 : la ligne avec batterie_pct = 150 est rejetée.

Figure 3 · Test du statut d'alerte « fermée »

```
UPDATE alerte SET statut = 'fermée' WHERE statut = 'traitee';
```



```
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -c "UPDATE alerte SET statut = 'fermée' WHERE statut = 'traitee';"
UPDATE 0
```

Résultat UPDATE 0 : aucune alerte n'a le statut « traitee » dans la base, la requête ne touche donc aucune ligne.

Étape 8.3 · Le trigger d'audit : qui a fait quoi

Figure 4 · Exécution de audit.sql

`psql -d lab8 -f audit.sql`

```
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -f audit.sql
CREATE FUNCTION
CREATE FUNCTION
DROP TRIGGER
CREATE TRIGGER
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:audit.sql:41: NOTICE:  trigger "journal_ajout_seul" for relation "journal_audit" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER
psql:audit.sql:46: NOTICE:  trigger "journal_sans_truncate" for relation "journal_audit" does not exist, skipping
DROP TRIGGER
CREATE TRIGGER
```

Création de qui(), audit_ligne() et journal_interdit(), puis des triggers sur recolte, utilisateur et journal_audit (journal_ajout_seul et journal_sans_truncate).

Figure 5 · Modification d'une récolte et lecture du journal

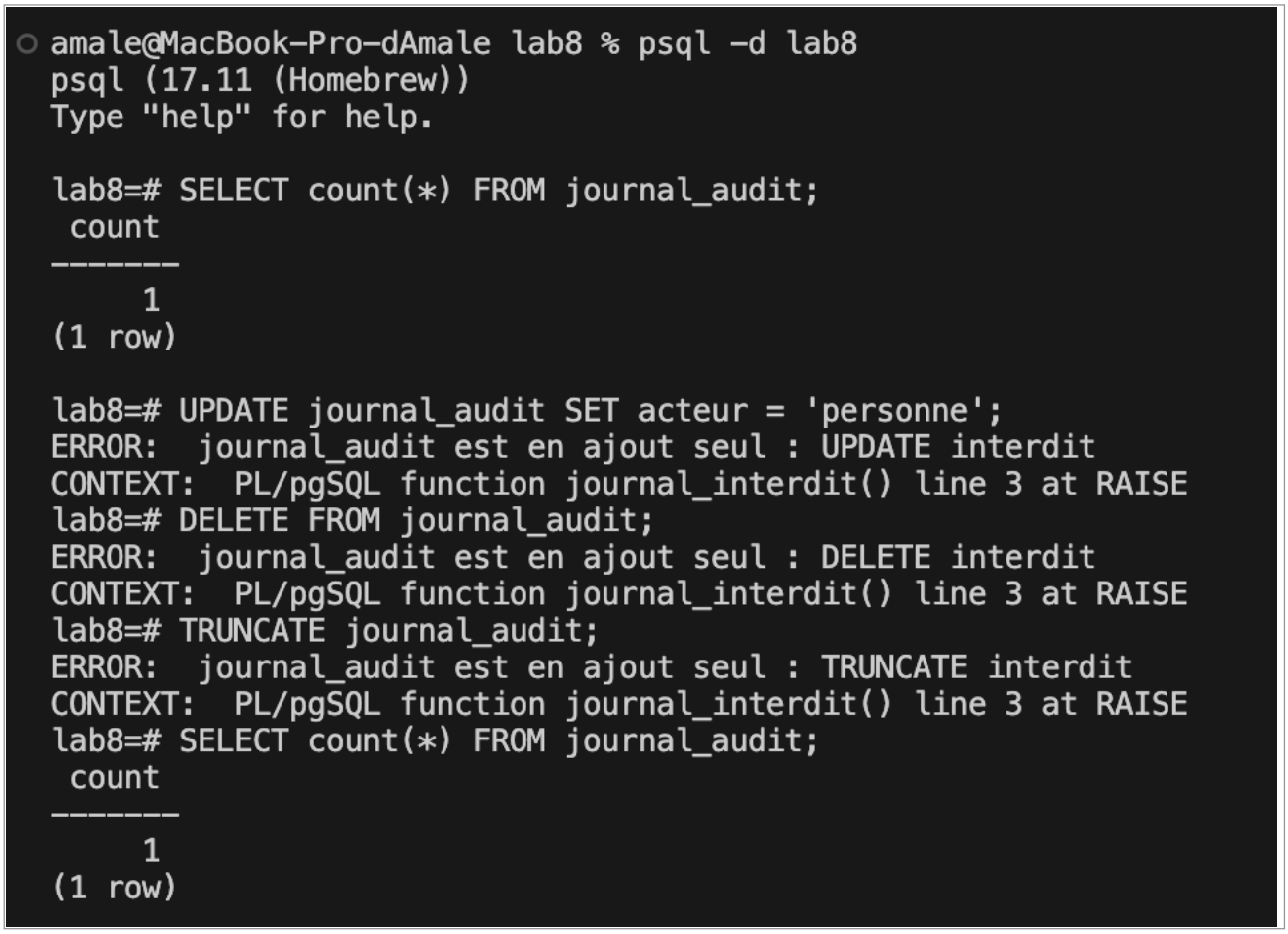
`UPDATE recolte SET quantite_kg = 14.0 ... ; SELECT ... FROM journal_audit ORDER BY id DESC LIMIT 1;`

```
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -c "UPDATE recolte SET quantite_kg = 14.0 WHERE ruche_id = (SELECT id FR
OM ruche WHERE nom = 'SN-CASA-014') AND quantite_kg = 12.50;"
UPDATE 1
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -c "SELECT acteur, table_cible, operation, avant ->> 'quantite_kg' AS av
ant, apres ->> 'quantite_kg' AS apres FROM journal_audit ORDER BY id DESC LIMIT 1;"
 acteur | table_cible | operation | avant | apres
--------+-------------+-----------+-------+-------
 amale  | recolte     | UPDATE    | 12.50 | 14.00
(1 row)
```

Le journal contient la modification : acteur amale, table recolte, opération UPDATE, valeur avant 12.50, valeur après 14.00.

Figure 6 · Tentatives d'effacement du journal

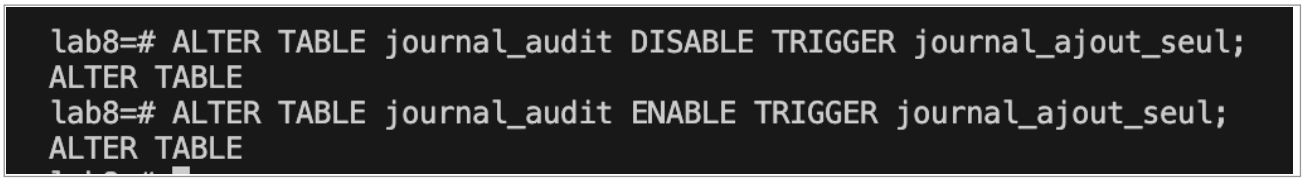
`UPDATE / DELETE / TRUNCATE journal_audit`



Trois refus (« journal_audit est en ajout seul ») et le nombre de lignes reste à 1.

Figure 7 · Désactivation du trigger par le propriétaire

`ALTER TABLE journal_audit DISABLE TRIGGER journal_ajout_seul; ... ENABLE TRIGGER ...;`



Le propriétaire peut désactiver puis réactiver le trigger : c'est la limite du dispositif, d'où la copie du journal hors de la base.

Étape 8.4 · Les rôles et les vues de moindre privilège

Figure 8 · Exécution de roles.sql

```
psql -d lab8 -f roles.sql
```

```
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -f roles.sql
DO
DO
INSERT 0 0
psql:roles.sql:40: ERROR:  database "queenbee" does not exist
REVOKE
REVOKE
REVOKE
psql:roles.sql:46: ERROR:  database "queenbee" does not exist
GRANT
GRANT
psql:roles.sql:54: NOTICE:  view "v_mes_recoltes" does not exist, skipping
psql:roles.sql:54: NOTICE:  view "v_mes_mesures" does not exist, skipping
psql:roles.sql:54: NOTICE:  view "v_mes_ruches" does not exist, skipping
psql:roles.sql:54: NOTICE:  view "v_recoltes_site" does not exist, skipping
psql:roles.sql:54: NOTICE:  view "v_ruches_site" does not exist, skipping
DROP VIEW
CREATE VIEW
psql:roles.sql:64: ERROR:  column r.code does not exist
LINE 2: SELECT r.id, r.code, r.site_id, r.latitude, r.longitude, r.d...
                     ^
HINT:  Perhaps you meant to reference the column "u.role".
psql:roles.sql:69: ERROR:  relation "v_mes_ruches" does not exist
LINE 4: WHERE m.ruche_id IN (SELECT id FROM v_mes_ruches);
                                           ^
psql:roles.sql:75: ERROR:  column rc.volume_kg does not exist
LINE 2: SELECT rc.id, rc.ruche_id, rc.date_recolte, rc.volume_kg, rc...
                                                    ^
psql:roles.sql:83: ERROR:  column r.code does not exist
LINE 2: SELECT r.id, r.code, r.site_id, a.nom AS apicultrice, a.tele...
                     ^
HINT:  Perhaps you meant to reference the column "u.role".
psql:roles.sql:91: ERROR:  column rc.volume_kg does not exist
LINE 2: SELECT rc.id, rc.ruche_id, rc.date_recolte, rc.volume_kg, rc...
                                                    ^
CREATE FUNCTION
REVOKE
GRANT
GRANT
psql:roles.sql:129: ERROR:  relation "v_mes_ruches" does not exist
psql:roles.sql:130: ERROR:  relation "v_mes_recoltes" does not exist
psql:roles.sql:132: ERROR:  relation "v_ruches_site" does not exist
psql:roles.sql:134: ERROR:  relation "synthese_site_jour" does not exist
GRANT
GRANT
```

Les groupes et les cinq personnes sont créés (blocs DO). Plusieurs erreurs apparaissent : le nom de base queenbee n'existe pas (la base s'appelle lab8), et certaines colonnes (ruche.code, recolte.volume_kg) et la vue synthese_site_jour n'existent pas dans ce schéma, ce qui empêche la création de certaines vues.

Figure 9 · Vérification des cinq personnes

`SELECT rolname FROM pg_roles WHERE rolname IN (...) ORDER BY rolname;`

```
amale@MacBook-Pro-dAmale lab8 % psql -d lab8 -c "SELECT rolname FROM pg_roles WHERE rolname IN ('f.ba', 'awa.diop',
coord.sn', 'gest.beeafrica', 'admin.beeafrica') ORDER BY rolname;"
     rolname
-----------------
 admin.beeafrica
 awa.diop
 coord.sn
 f.ba
 gest.beeafrica
(5 rows)
```

Les cinq rôles existent : admin.beeafrica, awa.diop, coord.sn, f.ba, gest.beeafrica.

Étape 8.6 · Tests de contournement

Test 1 · f.ba lit apicultrice directement

`SET ROLE "f.ba"; SELECT * FROM apicultrice;`

```
lab8=# SET ROLE "f.ba";
SELECT * FROM apicultrice;
SET
ERROR:  permission denied for table apicultrice
lab8=> RESET ROLE;
RESET
```

Refusé : permission denied for table apicultrice. Résultat attendu.

Vérification · la vue v_mes_ruches

`SELECT to_regclass('public.v_mes_ruches');`

```
lab8=# SELECT to_regclass('public.v_mes_ruches');
 to_regclass
-------------

(1 row)
```

Le résultat est vide : la vue v_mes_ruches n'existe pas, à cause des erreurs de la figure 8. Les tests 2 à 5, qui passent par cette vue, ne sont donc pas encore faisables.

Test 6 · f.ba modifie directement un volume

`SET ROLE "f.ba"; UPDATE recolte SET quantite_kg = 99;`

```
lab8=# SET ROLE "f.ba";
UPDATE recolte SET quantite_kg = 99;
SET
ERROR:  permission denied for table recolte
```

Refusé : permission denied for table recolte. Résultat attendu.

Test 7 · f.ba appelle valider_recolte

```
SET ROLE "f.ba"; SELECT valider_recolte(gen_random_uuid());
```

```
lab8=# SET ROLE "f.ba";
SELECT valider_recolte(gen_random_uuid());
SET
ERROR:  permission denied for function valider_recolte
```

Refusé : permission denied for function valider_recolte. Résultat attendu.

Test 8 · f.ba écrit une fausse ligne dans le journal

```
INSERT INTO journal_audit (acteur, table_cible, operation) VALUES ('coord.sn',
'recolte', 'UPDATE');
```

```
lab8=# SET ROLE "f.ba";
INSERT INTO journal_audit (acteur, table_cible, operation)
VALUES ('coord.sn', 'recolte', 'UPDATE');
SET
ERROR:  permission denied for table journal_audit
```

Refusé : permission denied for table journal_audit. Résultat attendu.

Test 9 · f.ba désactive le trigger d'audit

```
ALTER TABLE recolte DISABLE TRIGGER recolte_audit;
```

```
RESET
lab8=# SET ROLE "f.ba";
ALTER TABLE recolte DISABLE TRIGGER recolte_audit;
SET
ERROR:  must be owner of table recolte
```

Refusé : must be owner of table recolte. Résultat attendu.

Vérification · le coordinateur dans la table utilisateur

```
SELECT id, login, role, site_id FROM utilisateur WHERE role = 'coordinateur';
```

```
lab8=# SELECT id, login, role, site_id
FROM utilisateur
WHERE role = 'coordinateur';
                  id                  |   login    |     role     | site_id
--------------------------------------+------------+--------------+---------
 de7c05a4-81f3-46cd-a0fa-e261282fe687 | coord.casa | coordinateur | SN-CASA
(1 row)
```

Un seul coordinateur existe dans utilisateur : coord.casa (site SN-CASA). Le rôle PostgreSQL créé s'appelle, lui, coord.sn.

Test 12 (partie) · droit DELETE sur utilisateur

```
SELECT has_table_privilege(current_user, 'utilisateur', 'DELETE') AS
peut_supprimer;
```

```
lab8=> SELECT
    has_table_privilege(current_user, 'utilisateur', 'DELETE')
    AS peut_supprimer;
 peut_supprimer
----------------
 f
(1 row)
```

Résultat f (faux) : l'utilisateur courant ne peut pas supprimer de ligne dans utilisateur.